

Sustained Vertex Cover on Temporal Graphs

Junqiang Peng

University of Electronic Science and Technology of China
Chengdu, China
jqpeng0@foxmail.com

Jingyang Zhao

University of Electronic Science and Technology of China
Chengdu, China
jingyangzhao1020@gmail.com

Tian Bai

Department of Informatics, University of Bergen
Bergen, Norway
tian.bai@uib.no

Mingyu Xiao*

University of Electronic Science and Technology of China
Chengdu, China
myxiao@gmail.com

Abstract

We consider a novel vertex cover problem on temporal graphs, where the edges in the graph may change over time, and a vertex selected into the solution has a lifespan d . Specifically, a vertex selected at time t can cover all incident edges in graphs from time slot t to $t + d - 1$. This model effectively captures the scenario of monitoring communication links via secure nodes (monitors) with limited lifespan in a dynamic network. We provide a systematic study of this problem from both theoretical and practical perspectives. We analyze its computational complexity, develop approximation and online algorithms with tight ratios, and present a parameterized algorithm and a tight quadratic kernel under fixed d . Experimental results on random and real-world temporal networks demonstrate the effectiveness of our algorithms. We believe that our systematic study not only reveals the nature of the problem itself, but also paves the way for investigating the “sustained” version of other problems on temporal graphs.

CCS Concepts

• Theory of computation → Design and analysis of algorithms; • Networks → Network algorithms.

Keywords

Temporal Graphs, Vertex Cover, Approximation Algorithms, Online Algorithms, Parameterized Algorithms

ACM Reference Format:

Junqiang Peng, Tian Bai, Jingyang Zhao, and Mingyu Xiao. 2026. Sustained Vertex Cover on Temporal Graphs. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792250>

1 Introduction

As one of the most well-studied NP-hard problems on static graphs, the VERTEX COVER problem has broad applications, especially in link monitoring and surveillance in sensor networks [39], transport networks [19], social networks [28], etc. In a network, reliable

vertices can surveil all transmissions or detect link failures through incident edges [5, 29]. This makes VERTEX COVER a natural fit for such scenarios if one wants to minimize the number of activated surveillance vertices. However, most real-world networks are inherently *dynamic and change over time*, such as sensor networks, social networks, transportation networks, and various physical systems [25, 33]. Dynamic networks are commonly (and naturally) modeled using *temporal graphs*, where the vertex set is fixed while the edge set changes over time [26]. Specifically, given a static underlying graph, a temporal graph is defined by specifying the edge set that appears at each discrete time slot within its lifetime.

In the temporal setting, it is natural to consider the *continuous surveillance*, where every link must be surveilled at every moment it exists. As an example, in a dynamic intrusion detection system, each temporary connection between devices in a corporate network must be scanned for malicious activity exactly when established, as delayed detection could allow attackers to exploit ephemeral vulnerabilities. The need for this continuous surveillance is critical in Web-scale applications, such as suspicious transactions in financial networks or misinformation spreading over social networks.

In many real-world applications, the graph changes over time, and links need to be continuously surveilled by nodes. When a node is activated as a surveillance node, it typically maintains continuous surveillance capability throughout its *active period*, which would be limited due to constraints on energy or resources. Since the resources of a node can be replenished, a node can be activated multiple times. Given that activating a node consumes resources, one seeks to minimize the total number of times nodes are activated.

Although several temporal extensions of VERTEX COVER have been proposed and studied for diverse real-world applications [1, 14, 23, 36], none adequately models the above scenario.

Our Model. Motivated by this, we introduce and study a new variant of VERTEX COVER on temporal graphs, called the SUSTAINED VERTEX COVER problem (SVC). Before reviewing the related existing models, we first provide a clear description of our problem. In this problem, a vertex at time slot t (also called a *temporal vertex*) can be selected to cover all edges incident to it from time slot t to $t + d - 1$, where d is called the lifespan of a temporal vertex. The goal is to select a minimum number of temporal vertices to cover all edges at all time slots during the lifetime of a given temporal graph.

Related Works. Our work fits within the line of research on temporal extensions of classic graph problems [2, 8, 22, 24, 30–32, 37]. For a more comprehensive introduction, we refer to the

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792250>

tutorial by Gionis et al. [16]. Here, we focus on introducing some related existing models of vertex cover on temporal graphs.

Motivated by the need to summarize the interaction timelines of entities in social networks, Rozenshtein et al. [36] introduced the NETWORK UNTANGLING problem and its several variants, which have since spawned several follow-up works [10–12, 15]. Among all variants of NETWORK UNTANGLING proposed in [36], the most relevant one to our model is k -MINTIMELINE_∞. There are some similarities and subtle differences between this model and ours. In k -MINTIMELINE_∞, a temporal vertex cover is an assignment of time intervals to vertices, where each vertex can be assigned at most k time intervals (during which the vertex is considered “active”), such that for every edge e and for every time slot t where e appears, at least one of the time intervals assigned to one of the endpoints of e includes t . The definition of “cover” in k -MINTIMELINE_∞ is similar to ours, and the concept of “time interval” in this model aligns with the “sustained activity” in our model. However, k -MINTIMELINE_∞ aims to minimize the maximum length of assigned intervals, with each vertex being assigned a limited number of time intervals, whereas SVC can be viewed as minimizing the overall number of intervals with a bounded length.

The SLIDING WINDOW TEMPORAL VERTEX COVER problem (SW-TVC), introduced by Akrida et al. [1], is also relevant as it is motivated by network surveillance applications. In SW-TVC, given a temporal graph and an integer δ , an edge has to be “covered” at least once at every δ consecutive time steps, and each temporal vertex can only cover edges at a single time slot. The goal is to minimize the number of temporal vertices in the resulting cover. This is fundamentally different from our SVC model, where every edge should be covered at every time slot, and each edge temporal vertex can cover all edges over the subsequent d time slots. Hamm et al. [20] further investigate SW-TVC on sparse graphs, showing that SW-TVC is NP-hard even when $\delta = 2$ and the maximum degree of the underlying graph is 2.

Another variant studied in the literature, called MULTISTAGE VERTEX COVER [14, 23], is rather different from our model. Given a temporal graph, it aims to find a vertex cover of bounded size at each time slot, such that the sum of symmetric differences of every two consecutive vertex covers is upper-bounded [23] or the symmetric difference of two consecutive vertex covers is upper-bounded [14].

Our Contribution. Our contributions are multifaceted and can be summarized as follows:

- **Conceptual contribution.** We introduce SVC, a novel temporal variant of VERTEX COVER, which naturally extends the static VERTEX COVER problem to dynamic settings, with particular relevance to continuous surveillance in dynamic networks.
- **Complexity analysis.** We systematically study the computational complexity of SVC under various combinations of the lifespan d and the maximum degree Δ of the underlying graph (see Table 1). Notably, The NP-hardness proof for case $d = \Delta = 2$ is particularly interesting.
- **Algorithms.** On the algorithmic side, we present a 2-competitive online algorithm for SVC, which also serves as a 2-approximation algorithm in the offline setting. Additionally, we design a fixed-parameter tractable algorithm with respect to the solution size

	$\Delta = 1$	$\Delta = 2$	$\Delta \geq 3$
$d = 1$		P	NP-hard
$d \geq 2$	P	NP-hard	NP-hard

Table 1: The computational complexity of SVC with different settings on the lifespan d of the temporal vertices and the maximum degree Δ of the underlying graph.

k , and provide a kernelization with $O(dk^2)$ edges. Both the approximation ratio and the kernel size (for fixed d) are tight under standard complexity assumptions, matching the known bounds for the classic VERTEX COVER problem on static graphs.

- **Experimental evaluation.** Our experiments on both real-world and synthetic temporal graphs reveal that algorithms which invest more time in optimizing local decisions at each time slot—despite lacking theoretical guarantees—tend to yield higher-quality solutions in practice.

Some technical details and proofs (marked with ♣) are deferred to the appendix. All codes and data are available at <https://github.com/Pengjq0000000/SVC>.

2 Preliminary

A (static) graph $G = (V, E)$ consists of a set V of vertices and a set $E \subseteq \{uv \mid u, v \in V, u \neq v\}$ of edges. A vertex cover of graph G is a vertex set $S \subseteq V$ such that for every edge $uv \in E$, $S \cap \{u, v\} \neq \emptyset$. For every $i, j \in \mathbb{N}$, where $i < j$, we denote $[i, j] = \{i, i+1, \dots, j\}$ and $[j] = [1, j]$. The temporal graphs we work with are as follows.

DEFINITION 1 (TEMPORAL GRAPHS). An undirected temporal graph is denoted by $\mathcal{G} = (V, (E_i)_{i \in [T]})$, where V is a vertex set, $E_1, E_2, \dots, E_T$ are edge sets over V , and T is called the lifetime of $\mathcal{G}$.

We refer to each $t \in [T]$ as a time slot (or time point) of $\mathcal{G}$. A vertex $v \in V$ at time slot t , denoted by $\langle v, t \rangle$, is called a temporal vertex. Similarly, an edge $uv \in E_t$ at time slot t , denoted by $\langle uv, t \rangle$, is called a temporal edge. For every $\ell, r \in [T]$, where $\ell \leq r$, we denote by $W = [\ell, r]$ the time window with starting time slot ℓ and ending time slot r . Let Z be a set of time slots within the lifetime of $\mathcal{G}$. For a temporal vertex subset S , we use $S[Z] = \{\langle u, t \rangle \in S \mid t \in Z\}$ to denote the subset of all temporal vertices in S at some time slot in Z and $V(S) = \{u \mid \langle u, t \rangle \in S\}$ to denote the set of underlying vertices of S . We simply write $E[Z]$ to denote $\bigcup_{i \in Z} E_i$. The underlying graph of $\mathcal{G}$ is the static graph $(V, E[T])$. A temporal graph $\mathcal{G}$ is called temporal path if the underlying graph of $\mathcal{G}$ is a path.

Now we introduce the concept of “cover” used in this paper. A temporal vertex $\langle v, t \rangle$ d -sustainedly covers all edges incident on v during time window $[t, \min(t + d - 1, T)]$, i.e., edges $vu \in E_i$ with $i \in [t, \min(t + d - 1, T)]$. Correspondingly, a temporal edge $\langle uv, t \rangle$ can be d -sustainedly covered by a temporal vertex $\langle w, t_w \rangle$ if $w \in \{u, v\}$ and $t_w \in [\max(t - d + 1, 1), t]$. Here, the integer d is called the lifespan of a temporal vertex.

DEFINITION 2 (d -SUSTAINED VERTEX COVER). A d -sustained vertex cover of a temporal graph $\mathcal{G} = (V, (E_i)_{i \in [T]})$ is a set of temporal vertices $S \subseteq \{\langle v, t \rangle \mid v \in V, t \in [T]\}$ such that for each temporal edge in $\mathcal{G}$, it is d -sustainedly covered by some temporal vertex in S .

The main problems studied in this paper are as follows.

PROBLEM 1 (SUSTAINED VERTEX COVER (SVC)). *Given a temporal graph $\mathcal{G} = (V, (E_i)_{i \in [T]})$ and an integer $d \leq T$, compute a minimum d -sustained vertex cover of $\mathcal{G}$.*

Problem 1 is called the *offline version* of our optimization problem, since all the information of the temporal graph is given in advance. We also consider the following *online version* of the problem.

PROBLEM 2 (ONLINE-SVC). *A positive integer d and a vertex set V are given in advance. The input is a sequence of edge sets $E_1, E_2, \dots, E_T$ which arrive one by one. The goal is to compute a sequence of temporal vertex subsets $C_1, C_2, \dots, C_T$ such that for each $t \in [T]$, (1) $C_t \subseteq \{v, t\} \mid v \in V\}$; (2) $\bigcup_{i \in [t]} C_i$ is a d -sustained vertex cover of $\mathcal{G} = (V, (E_i)_{i \in [t]})$; (3) C_t should be computed after E_t arrives and before E_{t+1} arrives.*

An *online algorithm* for Online-SVC needs to maintain a d -sustained vertex cover for the already arrived temporal graph and does not know the edge sets at future time slots.

We also consider the following *parameterized version* of SVC. For basic concepts of parameterized complexity, see [13] and [4].

PROBLEM 3 (PARA-SVC). *Given a temporal graph $\mathcal{G} = (V, (E_i)_{i \in [T]})$, an integer $d \leq T$, and a parameter k , the goal is to determine if there is a d -sustained vertex cover of $\mathcal{G}$ with size at most k .*

3 Computational Complexity

SUSTAINED VERTEX COVER is trivially NP-hard, as it is equivalent to VERTEX COVER when $T = 1$. Therefore, in this section, we study the computational complexity of SVC with the restrictions that the lifespan d and the maximum degree Δ of the underlying graph are constants. The corresponding problem is denoted by (d, Δ) -SVC.

Since solving $(1, \Delta)$ -SVC is essentially to solve T independent instances of VERTEX COVER, we know that $(1, \Delta \leq 2)$ -SVC is polynomial-time solvable while $(1, \Delta \geq 3)$ -SVC becomes NP-hard as VERTEX COVER is NP-hard on graphs with maximum degree 3. Thus, our main focus is on the case $d \geq 2$. We begin with the following result for the case $\Delta = 1$.

THEOREM 1 (♣). *For every $d \geq 2$, $(d, 1)$ -SVC is polynomial-time solvable.*

3.1 Hardness on Temporal Paths

VERTEX COVER on static graphs of maximum degree 2 can be easily solved in polynomial time. In contrast, we show that $(2, 2)$ -SVC is NP-hard even when the underlying graph of the input temporal graph is a simple path. Then, we get the complete characterization of SVC in Table 1. To prove this result, we first introduce an intermediate problem called SQUARE COVERING IN BINARY MATRIX (SCBM) and show that it can be reduced to $(2, 2)$ -SVC, and then prove the NP-hardness of SCBM.

In an $r \times c$ binary matrix M , each entry is either 0 or 1. An entry with a value of 1 is called a *one-entry*. The entry (i, j) of M , denoted by $M[i, j]$, is the element in the i th row and j th column of M . Two entries (i_1, j_1) and (i_2, j_2) are *adjacent* if $|i_1 - i_2| \leq 1$ and $|j_1 - j_2| \leq 1$. We use $M[i_1, j_1; i_2, j_2]$ to indicate the *submatrix* induced from row i_1 to i_2 and column j_1 to j_2 , where $1 \leq i_1 \leq i_2 \leq r$ and $1 \leq j_1 \leq j_2 \leq c$.

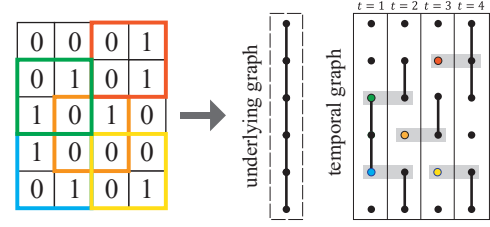


Figure 1: An example of reducing SCBM to $(2, 2)$ -SVC.

A submatrix of M with two rows and two columns is called a two-square. A set Q of two-squares is a *two-square covering* (or simply *covering*) of the binary matrix M if each 1-entry in M is included in some square in Q . The definition of SCBM is as follows:

PROBLEM 4 (SQUARE COVERING IN BINARY MATRIX (SCBM)). *Given an $r \times c$ binary matrix M with entries 0 and 1, compute a minimum two-square covering of M .*

On the left of Figure 1, we use five two-squares (with different colors) to cover all ones in the 5×4 binary matrix.

Reduction from SCBM to $(2, 2)$ -SVC. Let an $r \times c$ binary matrix M be an instance of SCBM. We construct an instance of $(2, 2)$ -SVC $\mathcal{G}_M = (V_M, (E_j)_{j \in [c]})$, where $V_M = \{1, 2, \dots, r+1\}$ and $E_j = \{(i, i+1) \mid M[i, j] = 1\}$ for $j \in [c]$. An illustration of the construction is shown in Figure 1. We can see that the underlying graph of the temporal graph is always a path. The five colored two-squares form an optimal covering of M , and these five colored temporal vertices in $\mathcal{G}_M$ form a minimum 2-sustained vertex cover.

LEMMA 1 (♣). *For any instance M of SCBM, there is a two-square covering with size at most s of M if and only if there is a 2-sustained vertex cover with size at most s of $\mathcal{G}_M$.*

NP-hardness of SCBM. We prove the NP-hardness of SCBM by a rather technical reduction from the known NP-hard problem MONOTONE 3-SAT [17]. MONOTONE 3-SAT is a specialization of the classical Boolean 3-Satisfiability problem, where each clause in a CNF-formula F contains only positive or only negative literals (termed *positive/negative clauses*) and contains at most three literals.

Given a MONOTONE 3-SAT instance F with variables $x_1, x_2, \dots, x_n$ and clauses $C_1, C_2, \dots, C_m$, we construct a binary matrix M_F such that F is satisfiable if and only if the minimum two-square covering of M_F has size $\leq \tau(M_F)$ (where $\tau(M_F)$ will be defined later).

An example of the whole reduction is given in Figure 2(a). For the sake of legibility, in all figures, colored matrix entries represent ones and uncolored matrix entries represent zeros. For each variable/clause, we will construct a variable/clause gadget (sets of one-entries in M_F ; non-gadget entries are zero). We now detail their construction and interconnections.

Variable gadget. For each variable x_i , we construct a *variable gadget* LP_i in M_F , forming an even-length “loop” of one-entries (i.e., $|LP_i| \bmod 2 = 0$). In Figure 2(a), the four colored “loops” in the middle are variable gadgets; each variable gadget occupies 4 columns and $16m + 2(m-1) + 6$ rows, with later protrusions added to connect with clause gadgets (which will be defined subsequently).

Labeling the loop entries $1, 2, \dots, |LP_i|$ counterclockwise, odd- and even-labeled entries are shaded dark and light, respectively. By

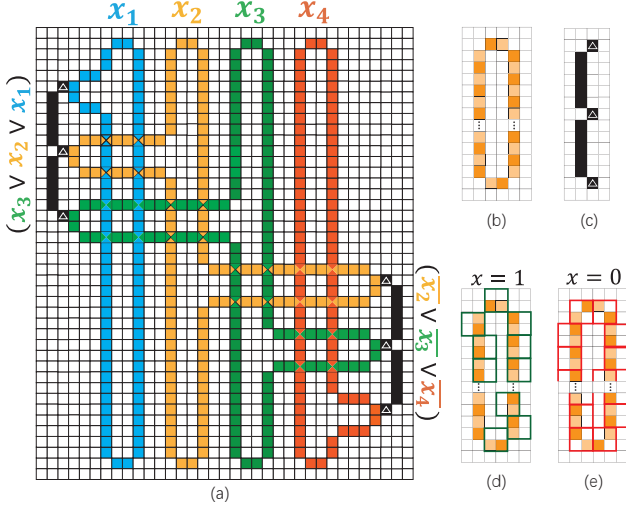


Figure 2: (a) an illustrative instance of SCBM that is reduced from the instance $F = (x_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_3 \vee \bar{x}_4)$ of MONOTONE 3-SAT; (b) a variable gadget; (c) a clause gadget, whose three special entries are marked with ‘△’; (d)(e) two kinds of optimal coverings for a variable gadget.

construction, a single two-square can cover at most two adjacent entries in LP_i , ensuring that the optimal covering size is $\frac{1}{2}|LP_i|$. Each variable gadget LP_i admits exactly two optimal coverings: (i) a light-dark alternating pattern (see Figure 2(d)), corresponding to assigning the variable true; (ii) a dark-light alternating pattern (see Figure 2(e)), corresponding to assigning the variable false. The n variable gadgets are mutually independent, meaning any optimal covering combines optimal coverings of individual loops. As a result, there are 2^n kinds of possible optimal coverings, matching the 2^n assignments of F .

Clause gadget. For each clause C_i , we construct a *clause gadget* B_i in M_F , consisting of 13 one-entries arranged in a 13×2 -submatrix. One column contains 10 entries and one column contains 3 *special entries* marked with a triangle in Figure 2(a) (see the two black graphics on the two sides).

The optimal covering size of a clause gadget is 7. However, removing any of the three special entries reduces this to 6. Our key idea is to plan variable and clause gadgets such that if a variable in C_i is set to true, the corresponding optimal covering also covers one special entry in B_i .

Interconnecting variables and clauses. For each variable x_i ($i \in [n]$) appearing in clause C_j ($j \in [m]$), we position the variable gadget LP_i adjacent to a special entry of clause gadget B_j . This ensures that an optimal covering of LP_i will cover a special entry of C_j if and only if x_i ’s assignment satisfies C_j . In the example shown in Figure 3(a), when x_1 and x_2 are assigned 0, their coverings affect the negative clause’s special entries but not the positive clause’s, while x_3 exhibits the opposite behavior.

We still need to address potential intersections between variable gadgets (Figure 2(a) and Figure 3(b)). Indeed, it does not affect the “independence” of optimal coverings: any combination of optimal

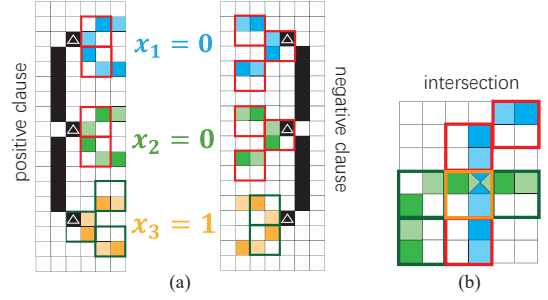


Figure 3: (a) an example showing how the variable and clause gadgets interconnect. The positive clause is $(x_1 \vee x_2 \vee x_3)$ and the negative clause is $(\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$; (b) an example of the intersection of two loops and their optimal coverings.

coverings for LP_i and LP_j forms an optimal covering for their union and vice versa. In Figure 3(b), when the blue loop (red squares) and green loop (green squares) are optimally covered, they share exactly one square (marked yellow) in the intersection area.

Definition of $\tau(M_F)$. Recall that $\tau(M_F)$ serves as a threshold value of the size of the minimum covering of M_F that indicates the satisfiability of F . Now we are poised to give the definition of $\tau(M_F)$ and the following lemma:

LEMMA 2 (♣). *For an arbitrary instance F of MONOTONE 3-SAT with n variables and m clauses, there is an assignment satisfying all clauses in F if and only if the size of the minimum two-square covering of M_F is at most $\tau(M_F)$, where*

$$\tau(M_F) = \frac{1}{2} \sum_{1 \leq i \leq n} |LP_i| - \sum_{1 \leq i < j \leq n} |LP_i \cap LP_j| + 6m.$$

With the above lemma, we have the following results.

THEOREM 2 (♣). *For every fixed $d \geq 2$, $(d, 2)$ -SVC is NP-hard, even on instances whose underlying graph is a path.*

THEOREM 3 (♣). *For every fixed $d \geq 2$ and $\Delta \geq 3$, (d, Δ) -SVC is NP-hard.*

4 Approximation and Online Algorithms

Since SVC is NP-hard, it is natural to consider approximation algorithms. Furthermore, we also consider the online version. In Online-SVC, the information on the temporal graphs (edge sets at each time slot) is given one by one, and we need to maintain the solution once a new edge set arrives. For online problems, we also compare the quality of the solution with the quality of the optimal solution for the offline version of the problem. This ratio is called the *competitive ratio*, while for offline problems, this ratio is called the *approximation ratio*.

We present two algorithm frameworks. One is designed for the offline scenario, and the other is also applicable to the online scenario. In both frameworks, we will invoke an algorithm (denoted by ComputeVC) as a subalgorithm to compute a vertex cover on static graphs. The varying implementations of ComputeVC lead to different algorithms, thereby instantiating the proposed frameworks. In this paper, we consider the following three frequently used implementations of ComputeVC:

Algorithm 1 Framework of algorithms for SVC**Input:** an instance $I = (\mathcal{G} = (V, (E_i)_{i \in [T]}), d)$.**Output:** a d -sustained vertex cover of $\mathcal{G}$.

- 1: $\theta \leftarrow$ integer sampled u.a.r. from $\{1, 2, \dots, d\}$;
- 2: Construct a partition $\mathcal{W}_\theta$ of $[T]$ with $p_\theta = \lceil \frac{T-\theta}{d} \rceil + 1$ time windows, where $|W_{\theta,1}| = \theta$, $|W_{\theta,2}| = \dots = |W_{\theta,p_\theta-1}| = d$, and $|W_{\theta,p_\theta}| = T - \theta - (p_\theta - 2)d$
- 3: $S \leftarrow$ an empty temporal vertex subset
- 4: **for** $i = 1$ **to** p_θ **do**
- 5: $H_i \leftarrow (V, E[W_{\theta,i}]); VC_i \leftarrow \text{ComputeVC}(H_i)$;
- 6: $S \leftarrow S \cup \{v, \ell_i \mid v \in VC_i\}$, where ℓ_i is the starting time slot of time window $W_{\theta,i}$;
- 7: **return** S

VC-Mini. This is to compute a minimum vertex cover, which may take exponential time due to the NP-hardness. There are several known fast solvers. We adopt an exact algorithm to check whether an optimal solution in each step can benefit the whole algorithm.

VC-Maxdeg. This algorithm is to iteratively pick a vertex of maximum degree, add it to the cover, and delete its edges. The implementation employing this strategy is denoted by Maxdeg. Maxdeg can compute a vertex cover with an approximation ratio $\Theta(\log n)$, where n is the number of vertices (with degree ≥ 1) in the graph [3]. Although the approximation ratio is not a constant, it performs well in practice [7].

VC-Edge. This algorithm repeatedly picks an edge not covered yet, adds both of its endpoints to the cover, and deletes edges incident to them.

4.1 Approximation Algorithms

Let $I = (\mathcal{G} = (V, (E_i)_{i \in [T]}), d)$ be an instance of SVC and W be a time window. We denote by $\mathcal{G}[W] = (V, (E_i)_{i \in W})$ the restricted temporal graph of $\mathcal{G}$ that only considers time slots in W and $I[W] = (\mathcal{G}[W], d)$ the restricted instance of I . A set of time windows $\{W_1, W_2, \dots, W_p\}$ is called a partition of $[T]$ if $W_1 \sqcup W_2 \sqcup \dots \sqcup W_p = [T]$, where $\sqcup$ denotes disjoint union. We have the following simple observation:

LEMMA 3. *Let S_i be an arbitrary feasible solution to $I[W_i]$ for each $i \in [p]$. Then $\bigcup_{i \in [p]} S_i$ is a feasible solution to I .*

Our main idea is then formalized as follows:

LEMMA 4 (MAIN IDEA). *Let S^* be an arbitrary optimal solution to I and $\{W_1, W_2, \dots, W_p\}$ be a partition of $[T]$. Let $\text{Apx}S_i$ be an α -approximate solution to $I[W_i]$ for each $i \in [p]$. Then, it holds that $\bigcup_{i \in [p]} \text{Apx}S_i$ is a $(\rho\alpha)$ -approximate solution to I , provided that $\sum_{i \in [p]} |\text{OPT}(I[W_i])| \leq \rho |S^*|$.*

PROOF. By Lemma 3, $\bigcup_{i \in [p]} \text{Apx}S_i$ is a feasible solution to I . In terms of the approximation ratio, we have $|\bigcup_{i \in [p]} \text{Apx}S_i| = \sum_{i \in [p]} |\text{Apx}S_i| \leq \sum_{i \in [p]} \alpha |\text{OPT}(I[W_i])| \leq \rho\alpha |S^*|$. $\square$

Our algorithm framework (presented in Algorithm 1) is motivated by the aforementioned idea. It first constructs a partition $\mathcal{W}_\theta = \{W_{\theta,1}, W_{\theta,2}, \dots, W_{\theta,p_\theta}\}$ of the lifetime T based on a randomly chosen integer $\theta \in [d]$. Next, it computes a solution for each restricted instance $I[W_{\theta,i}] (i \in [p_\theta])$ by utilizing the subalgorithm

ComputeVC as a component. The final solution for the input instance is obtained by taking the union of these solutions.

Following Lemma 4, we analyze the approximation ratio of the algorithm by evaluating ρ and α . We first address α .

LEMMA 5 ($\clubsuit$). *For an instance $I = (\mathcal{G} = (V, (E_i)_{i \in [T]}), d)$ of SVC with $T \leq d$, given an α -approximate minimum vertex cover of the underlying graph $G = (V, E[T])$, we can construct an α -approximate solution to I in polynomial time.*

Next, we conduct an analysis of ρ in Lemma 4.

LEMMA 6 ($\clubsuit$). *Let S^* be an arbitrary optimal solution to an instance $I = (\mathcal{G} = (V, (E_i)_{i \in [T]}), d)$, θ be an integer chosen from $\{1, 2, \dots, d\}$ uniformly at random, and $\mathcal{W}_\theta = \{W_{\theta,1}, W_{\theta,2}, \dots, W_{\theta,p_\theta}\}$ be the partition constructed in Algorithm 1. It holds that*

$$\mathbb{E} \left[\sum_{i \in [p_\theta]} |\text{OPT}(I[W_{\theta,i}])| \right] \leq (2 - 1/d) |S^*|.$$

Equipped with Lemma 5 and Lemma 6, we can apply Lemma 4 to achieve the following result.

THEOREM 4. *If ComputeVC computes an α -approximate minimum vertex cover, then Algorithm 1 computes a $(2\alpha - \alpha/d)$ -approximate solution for SVC in expectation.*

Algorithm 1 can be derandomized in a canonical manner: one can compute the results for all $\theta \in \{1, 2, \dots, d\}$ and select the minimum one among them as the solution. We remark that while VC-Mini computes optimal solutions for static graphs, Algorithm 1 combined with VC-Mini does not necessarily yield an optimal solution. According to Theorem 4, it achieves an approximation ratio of $2 - 1/d$. Since VC-Edge computes a 2-approximate minimum vertex cover, by Theorem 4, Algorithm 1 with VC-Edge is a $(4 - 2/d)$ -approximation algorithm. We will give a refined algorithm with a ratio of 2 in the next subsection.

4.2 Online Algorithms

For the online version, the edge set at each time slot is given one by one. We need to compute a vertex cover at each time slot t . To this end, we can get an online algorithm by restricting the length of each time window to 1 in Algorithm 2. However, the analysis might be different. We will further refine the algorithm and get a tight ratio of 2. Assume that at time slot t , an online algorithm has maintained a d -sustained vertex cover S_{t-1} for the already arrived temporal graph $\mathcal{G} = (V, (E_i)_{i \in [t-1]})$. When edge set E_t at the current time slot t arrives, the algorithm computes a temporal vertex subset $C_t \subseteq \{\langle v, t \rangle \mid v \in V\}$ such that $S_t = S_{t-1} \cup C_t$ is a d -sustained vertex cover of $\mathcal{G} = (V, (E_i)_{i \in [t]})$.

Note that some temporal edges in E_t might have already been d -sustained covered by S_t . Let $E_t^* = \{uv \in E_t \mid \forall \langle w, t_w \rangle \in S_{t-1}, w \neq \{u, v\} \text{ or } t - t_w \geq d\}$ be the set of remaining edges. At the current time slot, we need to select a vertex cover X_t of the current graph $G_t^* = (V, E_t^*)$. Based on this, we describe a general framework of online algorithms for Online-SVC in Algorithm 2.

When the framework uses VC-Mini, VC-Maxdeg, and VC-Edge to implement ComputeVC, the resulting algorithms are denoted by Mini, Maxdeg, and Edge, respectively. Interestingly, Algorithm Edge is an online algorithm with a competitive ratio of 2, which means that the size of the solution we obtained will not exceed twice

Algorithm 2 Framework of algorithms for Online-SVC

Input: an integer d , a vertex set V , and a sequence of edge sets $E_1, E_2, \dots, E_T$.

Output: a d -sustained vertex cover of $\mathcal{G} = (V, (E_i)_{i \in [T]})$.

```

1:  $S \leftarrow$  an empty temporal vertex subset
2: for  $t = 1$  to  $T$  do
3:    $E_t^* \leftarrow$  an empty edge set;
4:   for each  $uv \in E_t$  do
5:     if  $\langle uv, t \rangle$  is not covered by  $S$  then
6:       add edge  $uv$  to  $E_t^*$ 
7:    $G_t^* = (V, E_t^*)$ ;  $VC \leftarrow \text{ComputeVC}(G_t^*)$ ;
8:    $C_t \leftarrow \{\langle v, t \rangle \mid v \in VC\}$ ;
9:    $S \leftarrow S \cup C_t$ ;
10: return  $S$ 
```

the optimal solution of the offline problem. This also implies a 2-approximation algorithm for SVC.

THEOREM 5. *The competitive ratio of Edge is at most 2.*

PROOF. Let S be the solution computed by the algorithm and M be the set of temporal edges that are picked during the execution of the algorithm. We claim that for every pair of different temporal edges $\langle u_a v_a, t_a \rangle$ and $\langle u_b v_b, t_b \rangle$ in M , either $\{u_a, v_a\} \cap \{u_b, v_b\} = \emptyset$ or $|t_a - t_b| \geq d$. Suppose for the contradiction that there exists two temporal edges $\langle x_1 y_1, t_1 \rangle, \langle x_2 y_2, t_2 \rangle \in M$ such that $t_2 > t_1$, $\{x_1, y_1\} \cap \{x_2, y_2\} \neq \emptyset$, and $t_2 - t_1 < d$. Let $A = S[\max(1, t_2 - d + 1), t_2]$ be the set of vertices that can d -sustainedly cover some edges at time slot t_2 . Then, temporal vertices $\langle x_1, t_1 \rangle$ and $\langle y_1, t_1 \rangle$ should be in A , which implies that all edges incident to x_1 or y_1 at time slot t_2 would not be in M . This contradicts with $\langle x_2 y_2, t_2 \rangle \in M$ and $\{x_1, y_1\} \cap \{x_2, y_2\} \neq \emptyset$.

We have $|OPT(I)| \geq |M|$ since each temporal edge in M should be d -sustainedly covered by some temporal vertex, and every temporal vertex can only d -sustainedly cover at most one temporal edge in M . Clearly, $|S| = 2|M|$. Thus, we have $\frac{|S|}{|OPT(I)|} \leq 2$, which proves the theorem. $\square$

It is known that VERTEX COVER cannot be approximated within a factor $2 - \epsilon$ for any $\epsilon > 0$ in polynomial time under the Unique Games Conjecture (UGC) [27]. As VERTEX COVER is a special case of SVC, the competitive ratio (as well as the approximation ratio) of 2 achieved by Algorithm Edge is tight under UGC.

We remark that although this online algorithm has a better worst-case theoretical guarantee than Algorithm 1, it may not necessarily perform better in practice. In fact, Theorem 4 suggests that, in practical solving, the solution computed by Algorithm 1 might have better quality when ComputeVC returns a relatively small or even minimum vertex cover. This is also justified in our experiments.

5 Parameterized Algorithms

VERTEX COVER is a fundamental problem in parameterized complexity. It can be solved in $O(1.1996^n)$ time [38] or $O^*(1.2529^k)$ time [21], and has a kernel with $O(k^2)$ edges [6, 34], where n is the number of vertices and k is the solution size. It is natural to consider parameterized and kernelization algorithms for SUSTAINED VERTEX COVER parameterized by the solution size (i.e., our Problem 3

Para-SVC). In this section, we use n and m to denote the number of vertices and edges in the underlying graph, respectively. We first show that Para-SVC is fixed-parameter tractable.

THEOREM 6 (♣). *Para-SVC can be solved in time $O^*(2^k)$, where k is the solution size.*

Next, we focus on the kernelization algorithm for Para-SVC. Our algorithm will iteratively apply the following three reduction rules to simplify the instance.

REDUCTION RULE 1. *If v is an isolated vertex in the underlying graph, directly remove v .*

REDUCTION RULE 2. *If E_1 (resp., E_T) is empty, then remove E_1 (resp., E_T); If there are $d + 1$ consecutive empty edge sets $E_t, E_{t+1}, \dots, E_{t+d}$, remove E_{t+d} , and the lifetime T decreases by 1.*

The removal of an edge set E_t from $\mathcal{G} = (V, (E_i)_{i \in [T]})$ also removes the time slot t from the temporal graph, and all time slots after t are moved forward by one unit, resulting in a new temporal graph $\mathcal{G}' = (V', (E'_i)_{i \in [T-1]})$ where $V' = V$, $E'_i = E_i$ for each $i < t$, and $E'_i = E_{i+1}$ for each $i \geq t$.

REDUCTION RULE 3. *In each graph $G_t = (V, E_t)$, if there is a vertex v with more than $k + 1$ neighbors, remove an edge incident to v in G_t .*

LEMMA 7 (♣). *For each of the three reduction rules, an instance before applying it is a Yes-instance if and only if the instance after applying it is a Yes-instance.*

An instance is *reduced* if none of the three reduction rules can be applied. The last reduction rule is as follows.

REDUCTION RULE 4. *For a reduced instance, if $T > 2dk + 1$ or $m > dk(k + 1)$ or $n > 2dk(k + 1)$, report it as a No-instance.*

THEOREM 7 (♣). *Para-SVC admits a kernel with $O(dk)$ lifetime and $O(dk^2)$ vertices and edges.*

Unless $\text{coNP} \subseteq \text{NP/poly}$, VERTEX COVER parameterized by the solution size k admits no kernel with $O(k^{2-\epsilon})$ edges for any $\epsilon > 0$ [9]. As VERTEX COVER is a special case of Para-SVC with $T = 1$, this lower bound carries over to Para-SVC in terms of the number of edges in the underlying graph. Consequently, our kernel is tight for constant d under the assumption $\text{coNP} \not\subseteq \text{NP/poly}$.

6 Experimental Evaluation

We implement both our offline and online algorithms with three subalgorithms VC-Mini, VC-Maxdeg, and VC-Edge, and evaluate their performance (runtime and solution quality) on real-world and random temporal networks.

Setup. Algorithm Edge is implemented in $O(m)$ time, where m is the number of temporal edges. Algorithm Maxdeg is implemented in $O(n \log n + m)$ time, where n is the number of vertices. For Algorithm Mini, we use Gurobi Optimizer version 9.5.1 [18] to solve its integer programming model and return the best solution obtained by the solver within 2 seconds (which may not be a minimum vertex cover if it times out). Further details of the implementation can be found in Appendix D. However, we note that in most instances we test, it can return a minimum vertex cover within 2 seconds. All algorithms are implemented in C++17 and compiled using gcc

Table 2: Statistics of dataset, where $\mathcal{G} = (V, (E_i)_{i \in [T]})$ is the temporal graph and G is the underlying graph of $\mathcal{G}$.

	Dataset	$ V(\mathcal{G}) $	$ E(\mathcal{G}) $	$ E(G) $	T
real-world temporal	msg	1.9K	25.8K	13.8K	194
	email	1.0K	184.6K	16.1K	804
	mathoverflow	24.8K	294.3K	188.0K	2351
	askubuntu	159.3K	549.9K	455.7K	2614
	superuser	19.4K	857.6K	714.6K	2774
	wiki	1.1M	4.0M	2.8M	2321
	stackoverflow	2.6M	33.5M	28.2M	2775
peer-to-peer	p2p-Gnutella04	10.8K	18.2M	39.9K	500
	p2p-Gnutella08	6.3K	9.4M	20.7K	500
	p2p-Gnutella24	26.5K	29.7M	65.4K	500
	p2p-Gnutella30	36.7K	40.2M	88.3K	500
	p2p-Gnutella31	62.6K	67.3M	147.9K	500
random	gnp-100-0.1	100	247.4K	4.95K	500
	gnp-500-0.1	500	6.2M	124.75K	500
	gnp-1000-0.1	1.0K	24.9M	495K	500
	gnp-5000-0.01	5.0K	62.4M	12.4M	500

version 9.4.0 with the -O3 flag enabled. All experiments are conducted on a 64-core Intel(R) Xeon(R) Gold 6226R CPU @ 2.90GHz, equipped with 512 GB of memory and running a 64-bit Ubuntu 20.04 operation system.

Dataset. We consider three types of datasets, including real-world and generated temporal graphs, each of which is processed in a certain way or follows specific rules for generation. The first type contains seven real-world temporal graphs from the SNAP¹, which are social or message networks from different domains and cover a range of sizes across different orders of magnitude [35]. The second type is generated from static Internet networks. We select five peer-to-peer networks from SNAP with the number of edges ranging from 20K to 147.9K. The third type of dataset is random temporal graphs with vertex numbers $n = 100, 500, 1000$, and 5000. The details of the generation rules can be found in Appendix E.

The statistics of processed/generated datasets are presented in Table 2. We note that the graphs in the first two types of datasets are relatively sparse, and the randomly generated graphs are relatively dense when the number of vertices is large.

From our algorithmic framework, it can be observed that as the number of time slots increases, the running time of the algorithm will (linearly) increase. However, since the approximation ratio is independent of T , changes in T will not affect the solution quality. In our experiments, we also observed that the solution quality of the algorithm on instances with different values of T is almost the same. Thus, we only presented results for a single choice of $T = 500$.

Results for online algorithms. Table 3 shows the efficiency and accuracy of the (three) online algorithms on tested instances. We report the solution quality as the ratio of the solution size obtained by our algorithm to that of Algorithm Mini; the corresponding absolute values are available in our code repository. In all instances

of the first and second types of datasets, Mini computes a minimum vertex cover at each time slot within 2 seconds. On large random graphs, it times out and only outputs a heuristic solution (marked with ‘*’ in Table 3), which can be viewed as a high-quality heuristic for comparison. Nevertheless, Mini achieves the best solution quality among the three algorithms in almost all instances (with only one exception). On the other hand, its runtime is the longest, with an order of magnitude difference compared to the other two algorithms. Since Edge is a 2-approximation algorithm, half of its solution size provides a valid lower bound on the optimal value. Moreover, given that Mini and MaxDeg consistently outperform Edge in the experiment, their output solutions are experimentally within twice the optimum across all tested instances, especially on real-world instances. Next, we focus on the experimental results for Maxdeg and Edge, which have small runtimes.

In Figure 4, we compare the solution quality and runtimes of Maxdeg and Edge in some selected instances. The complete results for all instances are placed in Appendix F. In all instances, Maxdeg takes a longer time than Edge, but solution quality is better.

Results for approximation algorithms. When the framework (Algorithm 1) uses VC-Mini, VC-Maxdeg, and VC-Edge to implement ComputeVC, the resulting algorithms are denoted by Offline-Mini, Offline-Maxdeg, and Offline-Edge, respectively. Recall that Mini, Maxdeg, and Edge are the resulting algorithms when the framework of online algorithms for SVC (Algorithm 2) uses VC-Mini, VC-Maxdeg, and VC-Edge to implement ComputeVC.

We compare the quality of the solutions computed by the online and offline algorithm frameworks when using the same implementation of ComputeVC. In Table 4, we present the ratio of the solution sizes computed by the online algorithm to those computed by the offline algorithm in all instances. A ratio greater than 1 indicates that the offline algorithm performs better. From the table, we can see that in most of the tested instances, with the same implementation of ComputeVC, the offline algorithm generally produces better results. The improvement is especially significant when d is large. This is expected, as the offline algorithm utilizes more information.

7 Conclusion

In this paper, we propose a vertex cover problem on temporal graphs where each selected vertex has a lifespan on the temporal graph. This model has significant application prospects in networks and other real-life scenarios, particularly in simulating the selection of certain vertices to monitor dynamic networks with potentially disconnected edges. This paper comprehensively introduces a “sustained” problem on temporal graphs, which should be an interesting model worthy of in-depth exploration of more graph problems.

Acknowledgements

Junqiang Peng and Mingyu Xiao are funded by the National Natural Science Foundation of China under the grant 62372095. Tian Bai is funded by the European Union, GA#101126560; Bergen research and training program for future AI leaders across the disciplines, LEAD AI. Jingyang Zhao is funded by the National Natural Science Foundation of China under the grant 62502078 and the Postdoctoral Fellowship Program of CPSF under Grant Number GZC20251102.

¹Stanford Network Analysis Project: <https://snap.stanford.edu/data/index.html>

Table 3: Runtimes in seconds, and values within parentheses represent the solution quality, which is the ratio of the solution size output by the algorithm to the solution size output by the Algorithm Mini. The best results are highlighted in bold. Runtimes marked with ‘*’ indicate that Algorithm Mini reached the 2-second limit when solving the instance at each time slot.

Dataset	Results for $d = 2$			Results for $d = 5$			Results for $d = 20$			Results for $d = 50$			
	Mini	Maxdeg	Edge	Mini	Maxdeg	Edge	Mini	Maxdeg	Edge	Mini	Maxdeg	Edge	
real-world temporal	msg	1.37	0.01 (1.02)	0.00 (1.71)	1.30	0.01 (1.03)	0.00 (1.57)	1.17	0.01 (1.03)	0.00 (1.41)	1.07	0.00 (1.01)	0.00 (1.35)
	email	4.70	0.10 (1.02)	0.01 (1.61)	4.50	0.05 (1.04)	0.01 (1.42)	4.22	0.02 (1.06)	0.01 (1.28)	4.14	0.02 (1.07)	0.01 (1.24)
	mathoverflow	48.79	0.19 (1.01)	0.05 (1.72)	48.15	0.14 (1.03)	0.04 (1.58)	44.77	0.09 (1.08)	0.03 (1.44)	41.96	0.07 (1.12)	0.02 (1.40)
	askubuntu	250.98	0.42 (1.02)	0.09 (1.82)	245.66	0.32 (1.04)	0.07 (1.77)	229.79	0.22 (1.08)	0.07 (1.70)	224.26	0.18 (1.10)	0.06 (1.66)
	superuser	341.12	0.67 (1.01)	0.14 (1.82)	348.95	0.53 (1.03)	0.13 (1.76)	348.57	0.36 (1.07)	0.11 (1.69)	334.18	0.29 (1.10)	0.10 (1.66)
	wiki	669.13	2.23 (1.05)	0.39 (1.73)	571.20	1.46 (1.12)	0.32 (1.65)	489.34	0.81 (1.25)	0.25 (1.58)	463.60	0.57 (1.33)	0.23 (1.55)
	stackoverflow	5951.32	24.97 (1.01)	5.87 (1.75)	5523.30	17.93 (1.02)	5.47 (1.64)	5026.69	13.10 (1.06)	4.86 (1.53)	4709.69	10.48 (1.09)	4.68 (1.47)
peer-to-peer	p2p-Gnutella04	53.35	6.77 (1.02)	0.83 (1.64)	27.10	3.25 (1.03)	0.67 (1.64)	13.53	1.51 (1.03)	0.59 (1.63)	8.74	1.17 (1.03)	0.57 (1.63)
	p2p-Gnutella08	29.05	3.35 (1.01)	0.45 (1.74)	15.31	1.63 (1.02)	0.37 (1.74)	7.93	0.76 (1.02)	0.33 (1.73)	5.54	0.59 (1.02)	0.33 (1.73)
	p2p-Gnutella24	72.88	12.81 (1.01)	1.50 (1.74)	38.48	5.98 (1.02)	1.23 (1.74)	21.41	2.62 (1.02)	1.08 (1.73)	13.89	1.93 (1.02)	1.06 (1.73)
	p2p-Gnutella30	98.54	18.31 (1.01)	2.20 (1.76)	52.03	8.60 (1.02)	1.82 (1.76)	30.01	3.85 (1.02)	1.64 (1.75)	18.87	2.90 (1.02)	1.59 (1.75)
	p2p-Gnutella31	168.96	36.14 (1.02)	3.87 (1.79)	85.52	16.40 (1.02)	3.15 (1.79)	47.38	6.91 (1.02)	2.80 (1.78)	32.76	4.97 (1.02)	2.72 (1.78)
random	gnp-100-0.1	10.57	0.06 (1.03)	0.01 (1.23)	3.50	0.02 (1.01)	0.01 (1.13)	2.98	0.01 (1.00)	0.01 (1.05)	2.66	0.01 (1.00)	0.01 (1.01)
	gnp-500-0.1	*1013.37	1.09 (1.01)	0.19 (1.07)	*449.65	0.41 (1.01)	0.18 (1.05)	*37.93	0.23 (1.00)	0.18 (1.01)	*34.25	0.21 (1.00)	0.18 (1.00)
	gnp-1000-0.1	*1025.63	4.55 (1.00)	0.74 (1.04)	*874.45	1.63 (1.00)	0.73 (1.03)	*132.84	1.21 (1.00)	0.71 (1.01)	*31.58	0.91 (1.00)	0.73 (1.00)
	gnp-5000-0.01	*1047.70	13.98 (0.99)	1.97 (1.06)	*998.92	4.56 (1.00)	1.86 (1.04)	*310.37	2.56 (1.00)	1.82 (1.01)	*76.80	2.33 (1.00)	1.83 (1.00)

Table 4: The rounded ratio of the solution sizes computed by the online algorithm to those computed by the offline algorithm in all instances. In the table, ratios with actual values very close to 1 (between 0.995 and 1.004) are directly denoted by “>1” or “<1” to indicate their relationship with 1. In the table, cells containing ratios greater than 1 (which signify that the offline algorithm outperforms the online algorithm) are bolded.

Dataset	Ratios for $d = 2$			Ratios for $d = 5$			Ratios for $d = 20$			Ratios for $d = 50$			
	VC-Mini	VC-Maxdeg	VC-Edge	VC-Mini	VC-Maxdeg	VC-Edge	VC-Mini	VC-Maxdeg	VC-Edge	VC-Mini	VC-Maxdeg	VC-Edge	
real-world temporal	msg	1.02	1.02	1.38	1.06	1.07	1.38	1.12	1.12	1.32	1.14	1.13	1.27
	email	1.01	1.02	1.28	1.05	1.08	1.25	1.08	1.13	1.22	1.11	1.15	1.20
	mathoverflow	1.02	1.03	1.41	1.07	1.08	1.41	1.13	1.19	1.44	1.16	1.28	1.46
	askubuntu	1.02	1.03	1.61	1.04	1.07	1.65	1.07	1.14	1.67	1.09	1.19	1.66
	superuser	1.02	1.02	1.55	1.04	1.06	1.59	1.07	1.13	1.63	1.09	1.18	1.63
	wiki	1.02	1.05	1.58	1.05	1.15	1.61	1.09	1.32	1.63	1.12	1.44	1.64
	stackoverflow	1.02	1.03	1.46	1.07	1.08	1.49	1.12	1.17	1.50	1.13	1.23	1.49
peer-to-peer	p2p-Gnutella04	>1	>1	1.08	>1	1.01	1.08	>1	1.02	1.08	1.01	1.02	1.08
	p2p-Gnutella08	>1	1.01	1.08	>1	1.01	1.09	>1	1.01	1.09	>1	1.01	1.08
	p2p-Gnutella24	>1	1.01	1.09	>1	1.01	1.09	>1	1.02	1.09	1.01	1.02	1.09
	p2p-Gnutella30	>1	1.01	1.09	>1	1.01	1.09	>1	1.02	1.09	>1	1.02	1.09
	p2p-Gnutella31	>1	1.01	1.10	>1	1.01	1.10	>1	1.01	1.10	>1	1.01	1.10
random	gnp-100-0.1	0.97	0.97	1.04	0.97	0.97	1.03	0.99	0.99	1.02	1.01	1.01	1.01
	gnp-500-0.1	0.97	0.98	1.01	0.97	0.97	1.01	<1	<1	>1	>1	>1	>1
	gnp-1000-0.1	0.99	0.99	>1	0.98	0.98	>1	<1	<1	>1	>1	>1	>1
	gnp-5000-0.01	0.99	0.98	1.01	0.98	0.97	1.01	0.99	<1	>1	>1	>1	>1

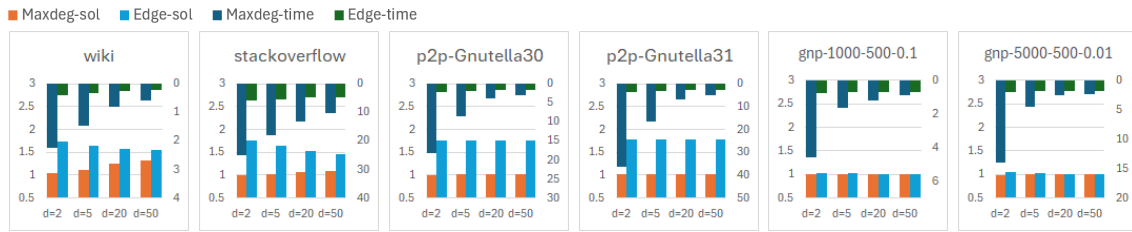


Figure 4: Solution quality and runtimes of Maxdeg and Edge on some instances. The left-hand side y -axis represents the solution quality, while the right-hand side y -axis represents the runtime in seconds.

References

- [1] Eleni C. Akrida, George B. Mertzios, Paul G. Spirakis, and Viktor Zamaraev. 2018. Temporal Vertex Cover with a Sliding Time Window. In *ICALP*, Vol. 107. 148:1–148:14.
- [2] Sebastian Buß, Hendrik Molter, Rolf Niedermeier, and Maciej Rymar. 2020. Algorithmic Aspects of Temporal Betweenness. In *KDD*. 2084–2092.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to algorithms*.
- [4] Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshantov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. 2015. *Parameterized Algorithms*. Springer. doi:10.1007/978-3-319-21275-3
- [5] Zuleyha Akusta Dagdeviren. 2023. A Metaheuristic Algorithm for Vertex Cover based Link Monitoring and Backbone Formation in Wireless Ad hoc Networks. *Expert Syst. Appl.* 213 (2023), 118919.
- [6] Frank K. H. A. Dehne, Michael R. Fellows, Frances A. Rosamond, and Peter Shaw. 2004. Greedy Localization, Iterative Compression, Modeled Crown Reductions: New FPT Techniques, an Improved Algorithm for Set Splitting, and a Novel $2k$ Kernelization for Vertex Cover. In *IWPEC*. 271–280.
- [7] François Delbot and Christian Laforest. 2010. Analytical and experimental comparison of six algorithms for the vertex cover problem. *ACM J. Exp. Algorithmics* 15 (2010).
- [8] Argyrios Deligkas, Eduard Eiben, and George Skretas. 2023. Minimizing Reachability Times on Temporal Graphs via Shifting Labels. In *IJCAI*. 5333–5340.
- [9] Holger Dell and Dieter van Melkebeek. 2014. Satisfiability Allows No Nontrivial Sparsification unless the Polynomial-Time Hierarchy Collapses. *J. ACM* 61, 4 (2014), 23:1–23:27.
- [10] Riccardo Dondi. 2023. Untangling temporal graphs of bounded degree. *Theor. Comput. Sci.* 969 (2023), 114040.
- [11] Riccardo Dondi and Manuel Lafond. 2025. An FPT algorithm for timeline cover. *J. Comput. Syst. Sci.* 154 (2025), 103679.
- [12] Riccardo Dondi and Alexandru Popa. 2025. Exact and approximation algorithms for covering timeline in temporal graphs. *Ann. Oper. Res.* 351, 1 (2025), 609–628.
- [13] Rodney G. Downey and Michael R. Fellows. 1999. *Parameterized Complexity*.
- [14] Till Fluschnik, Rolf Niedermeier, Valentin Rohm, and Philipp Zschoche. 2022. Multistage Vertex Cover. *Theory Comput. Syst.* 66, 2 (2022), 454–483.
- [15] Vincent Froese, Pascal Kunz, and Philipp Zschoche. 2022. Disentangling the Computational Complexity of Network Untangling. In *IJCAI*. 2037–2043.
- [16] Aristides Gionis, Lutz Oettershagen, and Ilie Sarpe. 2024. Mining Temporal Networks. In *WWW*. 1260–1263.
- [17] E. Mark Gold. 1978. Complexity of Automaton Identification from Given Data. *Inf. Control* 37, 3 (1978), 302–320.
- [18] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual.
- [19] Vasily V. Gusev. 2020. The vertex cover game: Application to transport networks. *Omega* 97 (2020), 102102.
- [20] Thekla Hamm, Nina Klobas, George B. Mertzios, and Paul G. Spirakis. 2022. The Complexity of Temporal Vertex Cover in Small-Degree Graphs. In *AAAI*. 10193–10201.
- [21] David G. Harris and N. S. Narayanaswamy. 2024. A Faster Algorithm for Vertex Cover Parameterized by Solution Size. In *STACS*, Vol. 289. 40:1–40:18.
- [22] Klaus Heeger, Danny Hermelin, George B. Mertzios, Hendrik Molter, Rolf Niedermeier, and Dvir Shabtay. 2021. Equitable Scheduling on a Single Machine. In *AAAI*. 11818–11825.
- [23] Klaus Heeger, Anne-Sophie Himmel, Frank Kammer, Rolf Niedermeier, Malte Renken, and Andrej Sajenko. 2021. Multistage graph problems on a global budget. *Theor. Comput. Sci.* 868 (2021), 46–64.
- [24] Anne-Sophie Himmel, Hendrik Molter, Rolf Niedermeier, and Manuel Sorge. 2017. Adapting the Bron-Kerbosch algorithm for enumerating maximal cliques in temporal graphs. *Soc. Netw. Anal. Min.* 7, 1 (2017), 35:1–35:16.
- [25] Petter Holme. 2018. Temporal Networks. In *Enc. Soc. Netw. Anal. Min.*
- [26] David Kempe, Jon M. Kleinberg, and Amit Kumar. 2000. Connectivity and inference problems for temporal networks. In *STOC*. 504–513.
- [27] Subhash Khot and Oded Regev. 2008. Vertex cover might be hard to approximate to within 2-epsilon. *J. Comput. Syst. Sci.* 74, 3 (2008), 335–349.
- [28] Dean Lusher, Johan Koskinen, and Garry Robins. 2013. *Exponential random graph models for social networks: Theory, methods, and applications*.
- [29] Liang Ma, Ting He, Kin K. Leung, Ananthram Swami, and Don Towsley. 2014. Inferring Link Metrics From End-To-End Path Measurements: Identifiability and Monitor Placement. *IEEE/ACM Trans. Netw.* 22, 4 (2014), 1351–1368.
- [30] Andrea Marino and Ana Silva. 2022. Coloring temporal graphs. *J. Comput. Syst. Sci.* 123 (2022), 171–185.
- [31] George B. Mertzios, Hendrik Molter, Rolf Niedermeier, Viktor Zamaraev, and Philipp Zschoche. 2023. Computing maximum matchings in temporal graphs. *J. Comput. Syst. Sci.* 137 (2023), 1–19.
- [32] George B. Mertzios, Hendrik Molter, and Viktor Zamaraev. 2019. Sliding Window Temporal Graph Coloring. In *AAAI*. 7667–7674.
- [33] Othon Michail and Paul G. Spirakis. 2018. Elements of the theory of dynamic networks. *Commun. ACM* 61, 2 (2018), 72.
- [34] George L. Nemhauser and Leslie E. Trotter Jr. 1974. Properties of vertex packing and independence system polyhedra. *Math. Program.* 6, 1 (1974), 48–61.
- [35] Ashwin Paranjape, Austin R. Benson, and Jure Leskovec. 2017. Motifs in Temporal Networks. In *WSDM*. 601–610.
- [36] Polina Rozenshtein, Nikolaj Tatti, and Aristides Gionis. 2021. The network-untangling problem: from interactions to activity timelines. *Data Min. Knowl. Discov.* 35, 1 (2021), 213–247.
- [37] Tiphaine Viard, Matthieu Latapy, and Clémence Magnien. 2016. Computing maximal cliques in link streams. *Theor. Comput. Sci.* 609 (2016), 245–252.
- [38] Mingyu Xiao and Hiroshi Nagamochi. 2017. Exact algorithms for maximum independent set. *Inf. Comput.* 255 (2017), 126–146.
- [39] Yasin Yigit, Vahid Khalilpour Akram, and Orhan Dagdeviren. 2021. Breadth-first search tree integrated vertex cover algorithms for link monitoring and routing in wireless sensor networks. *Comput. Networks* 194 (2021), 108144.

A Omitted Technical Details of Section 3

Proof of Theorem 1

PROOF. The algorithm is a greedy one, and the main idea is to do the following iteratively until there is no edge in the temporal graph: for an edge $uv \in E_t$ with the earliest possible time slot t , include temporal vertex $\langle u, t \rangle$ in the solution and remove all edges uv in $E_t, E_{t+1}, \dots, E_{\max(t+d-1, T)}$ if there exists. This is correct because at least one of the two temporal vertices $\langle v, t \rangle$ and $\langle u, t \rangle$ should be included in the solution to d -sustainedly cover the edge, and there is no other edge incident to u or v since $\Delta = 1$, i.e., the maximum degree of the underlying graph is one. The algorithm runs in $O(T + md)$ time, where $m = \sum_{i \in [T]} |E_i|$. $\square$

Proof of Lemma 1

PROOF. Assume M is an $r \times c$ binary matrix with $r, c \geq 2$. We use $SQ[r_i, c_i]$ to denote the 2×2 -square $M[r_i, c_i; r_i + 1, c_i + 1]$.

Assume that M admits a 2×2 -square covering $Q = \{SQ[r_i, c_i]\}_{i=1}^t$ with size $t \leq s$. One can construct a set of temporal vertices S in $\mathcal{G}_M$, where $S = \{\langle r_i + 1, c_i \rangle \mid SQ[r_i, c_i] \in Q\}$. Clearly, $|S| = |Q| = t \leq s$. Any edge in $\mathcal{G}_M$ corresponds to a 1-entry of M . Since Q is a 2×2 -square covering of M , the 1-entry must be covered by $SQ[r_{i'}, c_{i'}] \in Q$ for some $1 \leq i' \leq t$. Note that edges of $\mathcal{G}_M$ corresponding to the 1-entries in $SQ[r_{i'}, c_{i'}]$ are 2-sustainedly covered by $\langle r_{i'} + 1, c_{i'} \rangle$, with $\langle r_{i'} + 1, c_{i'} \rangle \in S$ by the construction of S . Thus, the edge is 2-sustainedly covered by $\langle r_{i'} + 1, c_{i'} \rangle \in S$. Therefore, S is a 2-sustained vertex cover with size at most s of $\mathcal{G}_M$.

Assume that $\mathcal{G}_M$ admits a 2-sustained vertex cover $S = \{\langle r_i, c_i \rangle\}_{i=1}^t$ with size $t \leq s$. One can construct a set of 2×2 -squares Q of M , where $Q = \{SQ[\max\{r_i - 1, 1\}, \min\{c_i, c - 1\}] \mid \langle r_i, c_i \rangle \in S\}$. Clearly, $|Q| = |S| = t \leq s$. Any 1-entry in M corresponds to an edge in $\mathcal{G}_M$. Since S is a 2-sustained vertex cover of $\mathcal{G}_M$, the edge must be 2-sustainedly covered by $\langle r_{i'}, c_{i'} \rangle \in S$ for some $1 \leq i' \leq t$. Note that 1-entries corresponding to the edges 2-sustainedly covered by $\langle r_{i'}, c_{i'} \rangle$ are contained in $SQ[\max\{r_i - 1, 1\}, \min\{c_{i'}, c - 1\}]$, with $SQ[\max\{r_i - 1, 1\}, \min\{c_{i'}, c - 1\}] \in Q$ by the construction of Q . Thus, the 1-entry must be covered by $SQ[\max\{r_i - 1, 1\}, \min\{c_{i'}, c - 1\}] \in Q$. Therefore, Q is a 2×2 -square covering with size at most s of M . $\square$

Proof of Lemma 2

PROOF. Suppose that F is satisfiable, i.e., there exists an assignment σ satisfying all clauses. For each variable x_i , based on its value under σ , we can construct $\lfloor \frac{1}{2} LP_i \rfloor$ squares with the corresponding pattern to cover the variable gadget LP_i . For each clause that is satisfied by the value of x_i under σ , the covering of LP_i

with the corresponding pattern also covers one of its special entries. As shown in Figure 3(b), each intersection of two variable gadgets can save one square. Thus, according to σ , we can construct $\frac{1}{2} \sum_{1 \leq i \leq n} |LP_i| - \sum_{1 \leq i < j \leq n} |LP_i \cap LP_j|$ squares to cover all the n variable gadgets. These squares also cover at least one special entry of each clause gadget since all clauses are satisfied under σ . Hence, we only need to construct $6m$ additional squares to cover all m clause gadgets. Thus, if F is satisfiable, the size of the optimal covering of M_F is $\tau(M_F)$. Next, we consider the opposite direction.

Suppose that there is a covering of M_F with size $\tau(M_F)$. According to our reduction, optimally covering a single variable gadget LP_i requires $\frac{1}{2}|LP_i|$ squares. Furthermore, optimally covering two variables gadgets LP_i and LP_j requires $\frac{1}{2}|LP_i| + \frac{1}{2}|LP_j| - |LP_i \cap LP_j|$, since each intersection of two variable gadgets saves one square. By analogy, optimally covering all n variables gadgets requires $c_{var} = \frac{1}{2} \sum_{1 \leq i \leq n} |LP_i| - \sum_{1 \leq i < j \leq n} |LP_i \cap LP_j|$ squares. That is, at least c_{var} squares are required to cover all variable gadgets.

Moreover, given that squares used to cover variable gadgets can only cover special entries of clause gadgets, at least $6m$ more squares are required to cover all the clause gadgets. Thus, in a covering of M_F with size $\tau(M_F) = c_{var} + 6m$, exactly c_{var} squares are used to cover the n variable gadgets, while the remaining $6m$ squares are used to cover the m clause gadgets. We denote by these two kinds of squares Q_v ($|Q_v| = c_{var}$) and Q_c ($|Q_c| = 6m$), respectively. So, Q_v corresponds to an assignment σ to F . Since all clause gadgets can be covered by Q_c after using Q_v to cover all variable gadgets, each clause gadget has at least one special entry covered by Q_c . Thus, each clause in F is satisfied by some variable under σ . $\square$

Proofs of Theorem 2 and Theorem 3

To prove these two theorems, we need the following lemma.

LEMMA 8. *For any $d \geq 1$ and $\Delta \geq 2$, there is a polynomial-time reduction from (d, Δ) -SVC to $(d+1, \Delta)$ -SVC.*

PROOF. Given an instance $\mathcal{I} = (\mathcal{G} = (V, (E_i)_{i \in [T]}), d)$ of (d, Δ) -SVC, we construct a new instance $\mathcal{I}' = (\mathcal{G}', d+1)$ of $(d+1, \Delta)$ -SVC, where $\mathcal{G}' = (V, (E_i)_{i \in [T + \lfloor T/d \rfloor]})$ is obtained from $\mathcal{G}$ by inserting empty time slots after every consecutive d time slots. See Figure 5 for an example. According to the construction of $\mathcal{G}'$, the t th time slot in $\mathcal{G}$ corresponds to the $f(t)$ th time slot in $\mathcal{G}'$, where

$$f(t) = t + \left\lfloor \frac{t-1}{d} \right\rfloor \quad \text{and} \quad f(t) \bmod (d+1) \neq 0. \quad (1)$$

Alternatively, the t' th time slot with $t' \bmod (d+1) \neq 0$ in $\mathcal{G}'$ corresponds to the $f^{-1}(t')$ th time slot in $\mathcal{G}$, where

$$f^{-1}(t') = t' - \left\lfloor \frac{t'}{d+1} \right\rfloor. \quad (2)$$

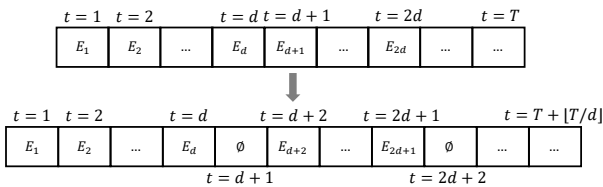


Figure 5: Inserting empty time slots.

Next, we prove that $\mathcal{G}$ admits a d -sustained vertex cover if and only if $\mathcal{G}'$ admits a $(d+1)$ -sustained vertex cover.

On one hand, assume that $\mathcal{G}$ admits a d -sustained vertex cover S . Then, one can get a set of temporal vertices S' in $\mathcal{G}'$ such that

$$S' = \{\langle v, f(t) \rangle \mid \langle v, t \rangle \in S\}. \quad (3)$$

Next, we prove that S' is a $(d+1)$ -sustained vertex cover in $\mathcal{G}'$.

Any edge $\langle uv, t' \rangle$ ($t' \bmod (d+1) \neq 0$) in $\mathcal{G}'$ corresponds to an edge $\langle uv, f^{-1}(t') \rangle$ in $\mathcal{G}$. Since S is a d -sustained vertex cover in $\mathcal{G}$, we assume that the edge $\langle uv, f^{-1}(t') \rangle$ is d -sustainedly covered by the temporal vertex $\langle v, f^{-1}(t'_1) \rangle \in S$ with

$$0 \leq f^{-1}(t') - f^{-1}(t'_1) \leq d-1. \quad (4)$$

By (1), (2), and (3), we have $f(f^{-1}(t'_1)) = t'_1$ and $\langle v, t'_1 \rangle \in S'$. It is sufficient to prove that the edge $\langle uv, t' \rangle$ is $(d+1)$ -sustainedly covered by the temporal vertex $\langle v, t'_1 \rangle$, i.e.,

$$0 \leq t' - t'_1 \leq d. \quad (5)$$

Since $0 \leq f^{-1}(t') - f^{-1}(t'_1)$ by (4), we can get $0 \leq t' - t'_1$ because the function $f^{-1}(\cdot)$ is monotone by (2). Moreover, if $t' - t'_1 > d$, we can get $t' - t'_1 \geq d+1$. Then, by (2), we can get

$$\begin{aligned} f^{-1}(t') - f^{-1}(t'_1) &= t' - t'_1 - \left(\left\lfloor \frac{t'}{d+1} \right\rfloor - \left\lfloor \frac{t'_1}{d+1} \right\rfloor \right) \\ &> (t' - t'_1) \cdot \frac{d}{d+1} - 1 \geq d-1, \end{aligned}$$

where the first inequality follows from that $x-1 < \lfloor x \rfloor \leq x$ holds for any x , and the second from $t' - t'_1 \geq d+1$. Since this contradicts with (4), we can get $t' - t'_1 \leq d$. Thus, (5) holds.

On the other hand, assume that $\mathcal{G}'$ admits a $(d+1)$ -sustained vertex cover S' . We may assume that S' contains no temporal vertices $\langle v, t \rangle$ with $t \bmod (d+1) = 0$. Otherwise, since the t th time slot is an empty slot, one can replace the temporal vertex with $\langle v, t+1 \rangle$ if $t < T + \lfloor T/d \rfloor$, or delete it if $t = T + \lfloor T/d \rfloor$. The new set is still a $(d+1)$ -sustained vertex cover in $\mathcal{G}'$. Then, one can get a set of temporal vertices S in $\mathcal{G}$ such that

$$S = \{\langle v, f^{-1}(t') \rangle \mid \langle v, t' \rangle \in S'\}. \quad (6)$$

Next, we show that S is a d -sustained vertex cover in $\mathcal{G}$.

Any edge $\langle uv, t \rangle$ in $\mathcal{G}$ corresponds to an edge $\langle uv, f(t) \rangle$ in $\mathcal{G}'$. Since S' is a $(d+1)$ -sustained vertex cover in $\mathcal{G}'$, we assume that the edge $\langle uv, f(t) \rangle$ is $(d+1)$ -sustainedly covered by the temporal vertex $\langle v, f(t_1) \rangle \in S'$ with

$$0 \leq f(t) - f(t_1) \leq d \quad \text{and} \quad t_1 \bmod (d+1) \neq 0. \quad (7)$$

By (1), (2), and (6), we have $f^{-1}(f(t_1)) = t_1$ and $\langle v, t_1 \rangle \in S$. It is sufficient to prove that the edge $\langle uv, t \rangle$ is d -sustainedly covered by the temporal vertex $\langle v, t_1 \rangle$, i.e.,

$$0 \leq t - t_1 \leq d-1. \quad (8)$$

Since $0 \leq f(t) - f(t_1)$ by (7), we can get $0 \leq t - t_1$ because the function $f(\cdot)$ is monotone by (1). Moreover, if $t - t_1 > d-1$, we can get $t - t_1 \geq d$. Then, by (1), we can get

$$\begin{aligned} f(t) - f(t_1) &= t - t_1 + \left(\left\lfloor \frac{t-1}{d} \right\rfloor - \left\lfloor \frac{t_1-1}{d} \right\rfloor \right) \\ &> (t - t_1) \cdot \frac{d-1}{d} + \frac{d+1}{d} \geq \frac{d^2+1}{d} > d, \end{aligned}$$

where the first inequality follows from that $x - 1 < \lfloor x \rfloor \leq x$ holds for any x , and the second from $t - t_1 \geq d$. Since this contradicts with (7), we can get $t - t_1 \leq d - 1$. Thus, (8) holds. $\square$

The NP-hardness of SCBM follows from Lemma 2 and the fact that MONOTONE 3-SAT is NP-hard [17]. Thus, by Lemma 1, we have (2, 2)-SVC is NP-hard. Then Theorem 2 and Theorem 3 follow from Lemma 8.

B Omitted Technical Details of Section 4

Proof of Lemma 5

PROOF. Denote by $vc^*(G)$ the size of the minimum vertex cover of G . Let S be an arbitrary feasible solution to $\mathcal{I}$. It is easy to see that $\{v \mid \langle v, t \rangle \in S\}$ is a vertex cover of G and thus $|OPT(\mathcal{I})| \geq vc^*(G)$.

Let VC be an arbitrary α -approximation minimum vertex cover of G , i.e., $|VC| \leq \alpha \cdot vc^*(G)$. We construct $S' = \{\langle v, 1 \rangle \mid v \in VC\}$, which is a feasible solution to $\mathcal{I}$ since $T \leq d$. Thus, we have $|OPT(\mathcal{I})| \leq |S'| = |VC| \leq \alpha \cdot vc^*(G)$. $\square$

Proof of Lemma 6

PROOF. Recall that $\mathcal{W}_\theta = \{W_{\theta,1}, \dots, W_{\theta,p_\theta}\}$ is a partition of $[T]$, where $|W_{\theta,i}| \leq d$ for each $i \in [p_\theta]$. We first describe the main idea. We have that $|OPT(\mathcal{I}[W_{\theta,i}])| \leq |S^*[W_{\theta,i-1}]| + |S^*[W_{\theta,i}]|$ for each $i \in [2, p_\theta]$. This is because for a temporal edge $\langle uv, t \rangle$ in $\mathcal{I}[W_{\theta,i}]$ ($i \in [2, p_\theta]$), it can only be d -sustainedly covered by a temporal vertex in $S^*[W_{\theta,i-1}]$ or $S^*[W_{\theta,i}]$. Consequently, we have

$$\mathbb{E} \left[\sum_{i \in [p_\theta]} |OPT(\mathcal{I}[W_{\theta,i}])| \right] \leq 2|S^*|.$$

To obtain an improved upper bound of $(2 - 1/d)|S^*|$, we observe that if $|W_{\theta,i-1}| = d$, the temporal vertices appearing at the starting (leftmost) time slot of $W_{\theta,i-1}$ are redundant, as their coverage range can not reach $W_{\theta,i}$. Hence, these temporal vertices can be excluded from the upper bound of $|OPT(\mathcal{I}[W_{\theta,i}])|$, and such a saving leads to the desired upper bound. Next, we complete the proof.

Denote by $L'_\theta = \{\ell_{\theta,i}\}_{i \in [p_\theta]}$ the set of starting time slot of time window $W_{\theta,i}$, where $\ell_{\theta,1} = 1$, $\ell_{\theta,2} = \theta + 1$, and $\ell_{\theta,i} = 1 + \theta + (i - 2)d$ for $i \in [3, p_\theta]$. Let $L_\theta = L'_\theta$ if $\theta = d$, and $L_\theta = L'_\theta \setminus \{\ell_{\theta,1}\}$ otherwise. It can be verified that $\bigcup_{\theta \in [d]} L_\theta = [T]$.

We have $|OPT(\mathcal{I}[W_{\theta,1}])| \leq |S^*[W_{\theta,1}]|$ and $|OPT(\mathcal{I}[W_{\theta,i}])| \leq |S^*[W_{\theta,i-1} \setminus \{\ell_{\theta,i-1}\}]] + |S^*[W_{\theta,i}]]|$ for $i \in [3, p_\theta]$. Moreover, we have $|OPT(\mathcal{I}[W_{\theta,2}])| \leq |S^*[W_{\theta,1} \setminus \{\ell_{\theta,1}\}]] + |S^*[W_{\theta,2}]]|$ if $|W_{\theta,1}| = d$ (i.e., $\theta = d$), and $|OPT(\mathcal{I}[W_{\theta,2}])| \leq |S^*[W_{\theta,1}]] + |S^*[W_{\theta,2}]]|$ otherwise. From this, for the case of $\theta = d$, we have

$$\begin{aligned} & \sum_{i \in [p_\theta]} |OPT(\mathcal{I}[W_{\theta,i}])| \\ & \leq (|S^*[W_{\theta,1}]]|) + \sum_{i \in [2, p_\theta]} (|S^*[W_{\theta,i-1} \setminus \{\ell_{\theta,i-1}\}]]| + |S^*[W_{\theta,i}]]|) \\ & = \sum_{i \in [p_\theta]} 2|S^*[W_i]]| - \left(\sum_{i \in [1, p_\theta-1]} |S^*[\{\ell_{\theta,i}\}]] \right) - |S^*[W_{\theta,p_\theta}]]| \\ & \leq \sum_{i \in [p_\theta]} 2|S^*[W_i]]| - \left(\sum_{i \in [1, p_\theta-1]} |S^*[\{\ell_{\theta,i}\}]] \right) - |S^*[\{\ell_{\theta,p_\theta}\}]]| \\ & \leq 2|S^*| - |S^*[L_\theta]]. \end{aligned}$$

Similarly, it can be verified that $\sum_{i \in [p_\theta]} |OPT(\mathcal{I}[W_{\theta,i}])| \leq 2|S^*| - |S^*[L_\theta]|$ also holds for the case $\theta \in [d - 1]$, according to the definition of L_θ . Denote $LB_\theta = \sum_{i \in [p_\theta]} |OPT(\mathcal{I}[W_{\theta,i}])|$ for brevity, we further have $\sum_{\theta \in [d]} LB_\theta \leq 2d|S^*| - \sum_{\theta \in [d]} |S^*[L_\theta]|$. Since $\bigcup_{\theta \in [d]} L_\theta = [T]$, we have $\sum_{\theta \in [d]} |S^*[L_\theta]| \geq |S^*|$. Thus,

$$\sum_{\theta \in [d]} LB_\theta \leq 2d|S^*| - |S^*| = (2d - 1)|S^*|.$$

Therefore, we have

$$\mathbb{E} \left[\sum_{i \in [p_\theta]} |OPT(\mathcal{I}[W_{\theta,i}])| \right] = \frac{1}{d} \sum_{\theta \in [d]} LB_\theta \leq (2 - \frac{1}{d})|S^*|.$$

$\square$

C Omitted Technical Details of Section 5

Proof of Theorem 6

PROOF. The idea of the algorithm is to iteratively do the following until $k < 0$ or there is no edge in the temporal graph: For an edge $vu \in E_t$ with the smallest possible time slot t , one of the two temporal vertices $\langle v, t \rangle$ and $\langle u, t \rangle$ should be selected in the solution set to d -sustainedly cover the edge. So, we can branch into two sub-branches: (1) remove all edges incident to v in $E_t, E_{t+1}, \dots, E_{t+d-1}$ and decrease k by 1; (2) remove all edges incident to u in $E_t, E_{t+1}, \dots, E_{t+d-1}$ and decrease k by 1. In each sub-branch, the parameter k decreases by 1. Then, the size of the search tree in the algorithm is $O^*(2^k)$. Therefore, the algorithm runs in $O^*(2^k)$ time. $\square$

Proof of Lemma 7

PROOF. Let $\mathcal{I} = ((\mathcal{G} = (V, (E_i)_{i \in [T]}), d), k)$ be the input instance such that v has more than $k + 1$ neighbors in G_t . Let $\mathcal{I}'$ be the instance after removing $vu \in E_t$ in $\mathcal{I}$ based on Reduction Rule 3, where v still has at least $k + 1$ neighbors in G'_t .

Clearly, a solution to $\mathcal{I}$ is still a solution to $\mathcal{I}'$. Next, we consider the other direction. Let S' be a d -sustained vertex cover of $\mathcal{I}'$ with the size at most k . In $\mathcal{I}'$, the degree of v is at least $k + 1$ in G'_t . A temporal vertex $\langle v, i \rangle$ with $t - d + 1 \leq i \leq t$ must be in S' since all edges incident to v in G_t needs to be d -sustainedly covered; otherwise, we need to use at least $k + 1$ temporal vertices to d -sustainedly cover all edges incident to v in G_t . Thus, the temporal vertex $\langle v, i \rangle$ can still d -sustainedly cover the edge vu in G_t in $\mathcal{I}$. Thus, the set S' is still a d -sustained vertex cover of $\mathcal{I}$. $\square$

Correctness of Reduction Rule 4

The correctness follows from the following lemma.

LEMMA. A reduced instance $((\mathcal{G}, d), k)$ with $T > 2dk + 1$ or $m > dk(k + 1)$ or $n > 2dk(k + 1)$ is a No-instance.

PROOF. Assume to the contrary that $T > d(2k + 1)$ and the instance is a Yes-instance with a solution S . We arbitrarily order the temporal vertices in S . For the i th temporal vertex $\langle v_i, t_i \rangle$, let x_i be the maximum positive integer such that $E_{t_i-x_i}, E_{t_i-x_i+1}, \dots, E_{t_i-1}$ are all empty. Note that x_i may not exist when E_{t_i-1} is non-empty. For this case, we simply let $x_i = 0$. By Reduction Rule 2 again, we know that $x_i \leq d$. Let $y_i = \min\{T, t_i + d - 1\}$ and $W_i = [t_i - x_i, y_i]$, and we have $|W_i| \leq 2d$. Each temporal vertex $\langle v_i, t_i \rangle \in S$ is associated

with a time window W_i . There are at most k time windows with length at most $2d$. If $T > 2dk$, then there is a time slot t^* not in any time window W_i . We can see that E_{t^*} is non-empty based on the following observation. If time slot t^* is not between any two time windows, then $E_{t^*} = \emptyset$ will be removed by Reduction Rule 2; if time slot t^* is between two time windows, then E_{t^*} should be non-empty by the definition of x_i . A temporal vertex $\langle v_i, t_i \rangle \in S$ can only d -sustainedly cover edges in the time window W_i . Thus, there is at least one edge in E_{t^*} , which is not d -sustainedly covered by any temporal vertex in S , a contraction to S being a solution.

In a reduced graph, each vertex in a static graph (V, E_i) has a degree bounded by $k + 1$. Thus, each temporal vertex can d -sustainedly cover at most $d(k + 1)$ edges. A solution of size at most k can d -sustainedly cover at most $dk(k + 1)$ temporal edges. Therefore, a reduced instance with more than $dk(k + 1)$ temporal edges is a No-instance. For a Yes-instance, the underlying graph has at most $dk(k + 1)$ edges. Then the underlying graph has at most $2dk(k + 1)$ vertices of degree ≥ 1 . Note that degree-0 vertices in the underlying graph would be deleted by Reduction Rule 2. Thus, if the underlying graph has more than $2dk(k + 1)$ vertices, the instance is also a No-instance. $\square$

D Implementation Details

Recall that in the framework of these three algorithms, at each time slot, we need to check if a temporal edge has been covered by some selected temporal vertices (Line 5 of Algorithm 2). This can be implemented in $O(1)$ time by using data structures Queue and Hashtable to maintain the solution.

In Algorithm Edge, as each edge can only be selected or deleted once. This can be implemented in $O(m)$ time, where m is the number of temporal edges. In Algorithm Maxdeg, we need to iteratively pick the vertex with maximum degree. This can be implemented in $O(n \log n + m)$ time using data structure Heap, where n is the number of vertices. For Algorithm Mini, we use Gurobi Optimizer version 9.5.1 [18] to solve its integer programming model and return the best solution obtained by the solver within 2 seconds (which may not be a minimum vertex cover).

E Descriptions of Data Generation Rules

For the first type, in the original datasets, edges are directed and associated with a timestamp in seconds, and there are few edges appearing per second, or even no edges at all. Therefore, we consider the edges that appear within a day (i.e., 86.4K seconds) as the edges at a single time slot. This ensures that the lifetime is appropriate and that there are not too few edges at each time slot. We also remove the direction of edges, duplicate edges, and self-loops.

For the second type, we generate temporal graphs based on them that can simulate scenarios where links disconnect due to faults or other reasons and then recover within a certain period of time. Let T be the lifetime of a temporal graph that we will generate. Our rule for generating the edge set at each time slot is as follows. The edge set E_1 at the first time slot contains all edges in the underlying graph, i.e., $E_1 = E$. For $t \in [2, T]$, the edge set E_t is generated based on E_{t-1} : each edge in E_{t-1} has a failure probability p and thus has a probability of $1 - p$ to appear in E_t . For each failure edge, i.e., each

edge in $E \setminus E_{t-1}$, it has a recovery probability q . We set $T = 500$, $p = 0.01$, and $q = 0.1$ when generating this type of temporal graphs.

The third type of dataset is random temporal graphs. We take lifetime $T = 500$ for all data and set vertex numbers $n = 100, 500, 1000$, and 5000. For each time slot t , we generate an edge set E_t using the Erdős-Rényi model: an edge between each distinct pair of n vertices is placed in E_t independently with probability p . We set $p = 0.1$ and $p = 0.01$ for graphs with $n \leq 1000$ and $n = 5000$, respectively.

F Omitted Experimental Results



Figure 6: Solution quality and runtimes of Maxdeg and Edge on all instances. The left-hand side y-axis represents the solution quality, while the right-hand side y-axis represents the runtime in seconds.

Figure 6 shows the solution quality and runtimes of Maxdeg and Edge on all instances. The solution quality is the ratio of the solution size output by the algorithm to the solution size output by the Algorithm Mini. In all instances, Maxdeg takes a longer time than Edge, but the solution quality is better.